LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Muhammad Nafis Putra **NIM. 2410817210021**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Nafis Putra
NIM : 2410817210021

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Raymond Hariyono	Erika Maulidiya, S.Kom., M.Kom.
NIM. 2310817210007	NIP. 200006192025062016

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 8
Source Code XML 8
Source Code Compose 12
B. Output Program 17
Output XML 17
Output Compose 19
C. Pembahasan 21
Pembahasan XML 21
Pembahasan Compose 23
SOAL 2 26
SOAL 3 28
Daftar Pustaka 29
Tautan Git 30

DAFTAR GAMBAR

Gambar 1. Screenshot Contoh Soal 1 6
Gambar 2. Screenshot Contoh Soal 1 7
Gambar 3. Screenshot Contoh Soal 1 7
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML Tampilan awal 17
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML Dropdown 17
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML Round Tip off 18
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML Round Tip on 18
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose Tampilan awal 19
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose Dropdown 19
Gambar 10. Screenshot Hasil Jawaban Soal 1 Compose Round Tip off 20
Gambar 11. Screenshot Hasil Jawaban Soal 1 Compose Round Tip on 20

DAFTAR TABEL

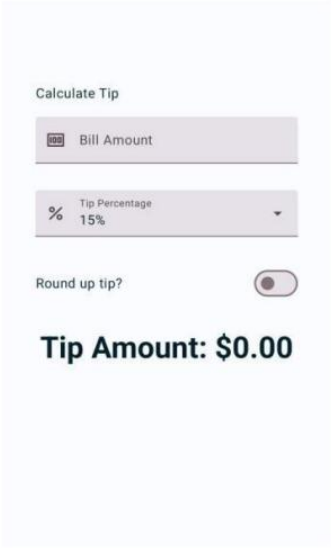
Tabel 1. Source Code Jawaban Soal 1 XML 8
Tabel 2. Source Code Jawaban Soal 1 XML 10
Tabel 3. Source Code Jawaban Soal 1 Compose 12

SOAL 1

Soal Praktikum:

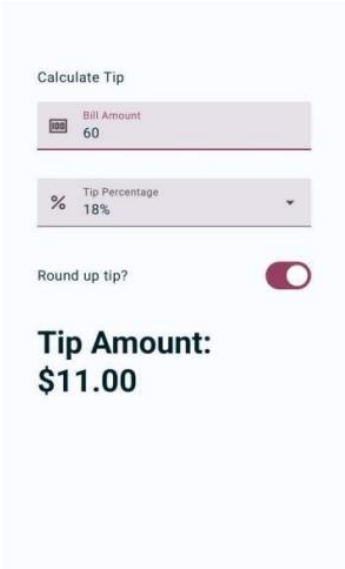
Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Screenshot Contoh Soal 1

Gambar 2. Screenshot Contoh Soal 1



Gambar 3. Screenshot Contoh Soal 1

A. Source Code

Source Code XML

MainActivity.kt

Tabel 1. Source Code Jawaban Soal 1 XML

```
package com.example.tip

import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.AutoCompleteTextView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.core.widget.doOnTextChanged
import com.google.android.material.switchmaterial.SwitchMaterial
import com.google.android.material.textfield.TextInputEditText
import java.text.NumberFormat
import java.util.Locale
import kotlin.math.ceil

class MainActivity : AppCompatActivity() {

    private lateinit var etBillAmount: TextInputEditText
    private lateinit var spinnerTipPercentage: AutoCompleteTextView
    private lateinit var switchRoundUp: SwitchMaterial
    private lateinit var tvTipAmount: TextView

    private var tipPercentageStr = "15%"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        etBillAmount = findViewById(R.id.etBillAmount)
        spinnerTipPercentage = findViewById(R.id.spinnerTipPercentage)
        switchRoundUp = findViewById(R.id.switchRoundUp)
        tvTipAmount = findViewById(R.id.tvTipAmount)

        if (savedInstanceState != null) {
            tipPercentageStr = savedInstanceState.getString("SAVED_TIP_PERCENT", "15%")
        }

        val tipOptions = listOf("15%", "18%", "20%")
        spinnerTipPercentage.post {
```

```
            val adapter = ArrayAdapter(this, android.R.layout.simple_dropdown_item_1line, tipOptions)
            spinnerTipPercentage.setAdapter(adapter)
            spinnerTipPercentage.setText(tipPercentageStr, false)
        }

        etBillAmount.doOnTextChanged { _, _, _, _ ->
            calculateTip()
        }
        spinnerTipPercentage.setOnItemClickListener { _, _, position, _ ->
            tipPercentageStr = tipOptions[position]
            calculateTip()
        }
        switchRoundUp.setOnCheckedChangeListener { _, _ ->
            calculateTip()
        }
        calculateTip()
    }

    override fun onSaveInstanceState(outState: Bundle) {
        super.onSaveInstanceState(outState)
        outState.putString("SAVED_TIP_PERCENT", tipPercentageStr)
    }
    private fun calculateTip() {
        val amountInput = etBillAmount.text.toString()
        val roundUp = switchRoundUp.isChecked

        val amount = amountInput.toDoubleOrNull() ?: 0.0
        val tipPercent = tipPercentageStr.replace("%", "").toDoubleOrNull() ?: 0.0

        var tip = (tipPercent / 100) * amount
        if (roundUp) {
            tip = ceil(tip)
        }

        val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)
        tvTipAmount.text = getString(R.string.tip_amount, formattedTip)
    }
}
```

activity_main.xml

Tabel 2. Source Code Jawaban Soal 1 XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/andro
id"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center_vertical"
    android:orientation="vertical"
    android:padding="32dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="24dp"
        android:text="@string/calculate_tip"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textColor="#000000"
        android:fontFamily="sans-serif-light"/>


<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.MaterialComponents.TextInputLayout
.FilledBox"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="24dp"
        android:hint="@string/bill_amount"
        app:startIconDrawable="@drawable/ic_money"
app:boxBackgroundColor="@color/surface_variant">


<com.google.android.material.textfield.TextInputEditTex
t
            android:id="@+id/etBillAmount"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="numberDecimal"
            android:singleLine="true" />
```

```
</com.google.android.material.textfield.TextInputLayout>


<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropdownMenu"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="24dp"
        android:hint="@string/tip_percentage"
        app:startIconDrawable="@drawable/ic_percent"
app:boxBackgroundColor="@color/surface_variant">

        <AutoCompleteTextView
            android:id="@+id/spinnerTipPercentage"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="none"
            android:importantForAccessibility="no"
android:popupBackground="@color/surface_variant"/>
</com.google.android.material.textfield.TextInputLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="24dp"
        android:gravity="center_vertical"
        android:orientation="horizontal">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="@string/round_up_tip"
            android:textSize="16sp" />


<com.google.android.material.switchmaterial.SwitchMaterial
            android:id="@+id/switchRoundUp"
```

```
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />
    </LinearLayout>

    <TextView
        android:id="@+id/tvTipAmount"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center_horizontal"
        android:textSize="32sp"
        android:textColor="#000000"
        android:text="@string/tip_amount"
        android:fontFamily="sans-serif-black"/>

</LinearLayout>
```

Source Code Compose

MainActivity.kt

Tabel 3. Source Code Jawaban Soal 1 Compose

```
package com.example.tip

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material3.*
import androidx.compose.runtime.*
import
androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import java.text.NumberFormat
import java.util.Locale
import kotlin.math.ceil

class MainActivity : ComponentActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            TipCalculatorScreen()
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TipCalculatorScreen() {
    var amountInput by rememberSaveable { mutableStateOf("") }
    var tipPercentageStr by rememberSaveable { mutableStateOf("15") }
    var roundUp by rememberSaveable { mutableStateOf(false) }
    var expanded by rememberSaveable { mutableStateOf(false) }

    val tipOptions = listOf("15", "18", "20")

    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tipPercent = tipPercentageStr.toDoubleOrNull() ?: 0.0

    var tip = (tipPercent / 100) * amount
    if (roundUp) {
        tip = ceil(tip)
    }

    val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)

    Column(
        modifier = Modifier
            .padding(32.dp)
            .fillMaxSize(),
        verticalArrangement = Arrangement.spacedBy(24.dp, Alignment.CenterVertically),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
```

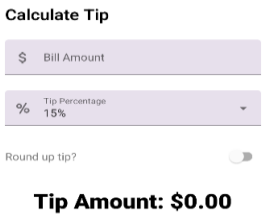
```
            text                                        =
stringResource(R.string.calculate_tip),
            fontSize = 24.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.align(Alignment.Start)
        )

        TextField(
            value = amountInput,
            onValueChange = { amountInput = it },
            label                   =                   {
Text(stringResource(R.string.bill_amount)) },
            leadingIcon = {
                Icon(
                    painter   =   painterResource(id   =
R.drawable.ic_money),
                    contentDescription = null
                )
            },
            keyboardOptions                             =
KeyboardOptions(keyboardType = KeyboardType.Number),
            singleLine = true,
            modifier = Modifier.fillMaxWidth(),
            colors = TextFieldDefaults.colors(
                focusedContainerColor                   =
MaterialTheme.colorScheme.surfaceVariant,
                unfocusedContainerColor                 =
MaterialTheme.colorScheme.surfaceVariant,
                focusedIndicatorColor                   =
MaterialTheme.colorScheme.primary,
                unfocusedIndicatorColor                 =
MaterialTheme.colorScheme.onSurfaceVariant
            )
        )

        ExposedDropdownMenuBox(
            expanded = expanded,
            onExpandedChange = { expanded = !expanded }
        ) {
            TextField(
                value                                   =
stringResource(R.string.percentage_symbol,
tipPercentageStr),
                onValueChange = { },
                readOnly = true,
                label                 =                 {
Text(stringResource(R.string.tip_percentage)) },
```

```
                leadingIcon = {
                    Icon(
                        painter = painterResource(id = R.drawable.ic_percent),
                        contentDescription = null,
                        modifier = Modifier.size(18.dp)
                    )
                },

                trailingIcon = { ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded) },
                modifier = Modifier
                    .menuAnchor()
                    .fillMaxWidth(),
                colors = TextFieldDefaults.colors(
                    focusedContainerColor = MaterialTheme.colorScheme.surfaceVariant,
                    unfocusedContainerColor = MaterialTheme.colorScheme.surfaceVariant,
                    focusedIndicatorColor = MaterialTheme.colorScheme.primary,
                    unfocusedIndicatorColor = MaterialTheme.colorScheme.onSurfaceVariant
                )
            )

            ExposedDropdownMenu(
                expanded = expanded,
                onDismissRequest = { expanded = false }
            ) {
                tipOptions.forEach { option ->
                    DropdownMenuItem(
                        text = { Text("$option%") },
                        onClick = {
                            tipPercentageStr = option
                            expanded = false
                        }
                    )
                }
            }
        }

        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
```

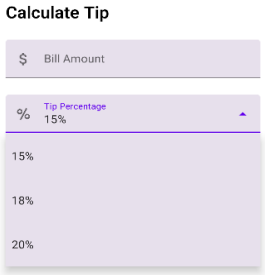
```
            horizontalArrangement                                   = Arrangement.SpaceBetween
        ) {
            Text(text                                               = stringResource(R.string.round_up_tip))
            Switch(
                checked = roundUp,
                onCheckedChange = { roundUp = it }
            )
        }

        Spacer(modifier = Modifier.height(16.dp))

        Text(
            text  =  stringResource(R.string.tip_amount, formattedTip),
            fontSize = 32.sp,
            fontWeight = FontWeight.Bold,
            modifier                                                = Modifier.align(Alignment.CenterHorizontally)
        )
    }
}
```

B. Output Program

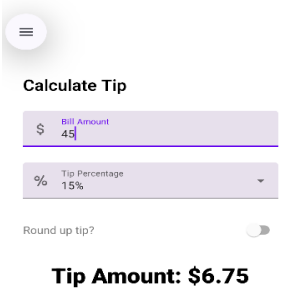
Output XML



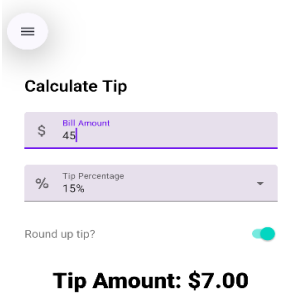
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML Tampilan awal



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML Dropdown

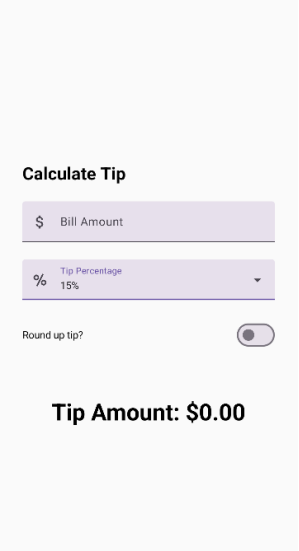


Gambar 6. Screenshot Hasil Jawaban Soal 1 XML Round Tip off

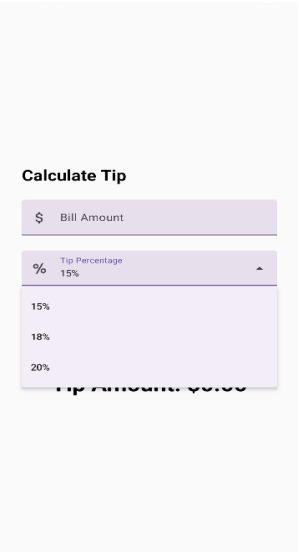


Gambar 7. Screenshot Hasil Jawaban Soal 1 XML Round Tip on

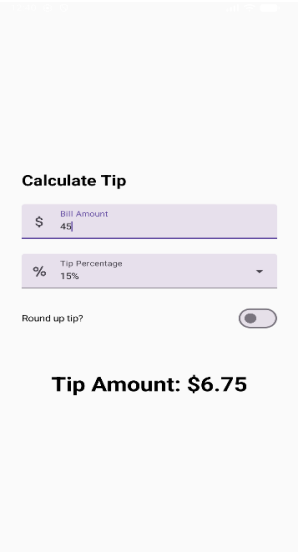
Output Compose



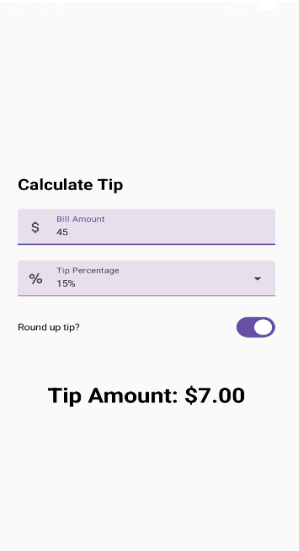
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose Tampilan awal



Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose Dropdown



Gambar 10. Screenshot Hasil Jawaban Soal 1 Compose Round Tip off



Gambar 11. Screenshot Hasil Jawaban Soal 1 Compose Round Tip on

C. Pembahasan

Pembahasan XML

MainActivity.kt:

Kode ini menggunakan metode perancangan XML. Dimulai dari baris ke-1 file Kotlin yang mendefinisikan package com.example.tip sebagai lokasi kelas dalam proyek. Baris ke-3 sampai baris ke-13 berisi import library seperti TextView, AutoCompleteTextView, SwitchMaterial, TextInputEditText, serta NumberFormat, Locale, dan ceil yang digunakan dalam proses perhitungan dan format nilai tip. Pada baris ke-15 dideklarasikan kelas MainActivity : AppCompatActivity() sebagai activity utama. Baris ke-17 sampai baris ke-20 mendeklarasikan variabel etBillAmount, spinnerTipPercentage, switchRoundUp, dan tvTipAmount yang akan terhubung dengan komponen pada file XML. Baris ke-22 mendefinisikan tipPercentageStr = "15%" sebagai nilai awal persentase tip. Pada kode ini menggunakan LinearLayout karena untuk aplikasi belum termasuk aplikasi yang kompleks, jadi akan lebih efisien apabila menggunakan LinearLayout karena diatur menggunakan Vertikal dan Horizontal.

Pada baris ke-24 dimulai fungsi onCreate(savedInstanceState: Bundle?), kemudian baris ke-26 memanggil setContentView(R.layout.activity_main) untuk menghubungkan layout XML. Pada file XML, baris ke-2 sampai baris ke-8 merupakan <LinearLayout> sebagai root dengan orientasi vertikal, yang dipilih karena struktur tampilan disusun secara berurutan dari atas ke bawah seperti judul, input, pilihan, switch, dan hasil, sehingga penggunaan layout ini lebih sederhana dan sesuai tanpa membutuhkan pengaturan posisi yang kompleks. Baris ke-10 sampai baris ke-18 merupakan <TextView> yang menampilkan judul dengan teks @string/calculate_tip.

Pada baris ke-20 sampai baris ke-35 pada XML terdapat <TextInputLayout> pertama yang membungkus input jumlah tagihan. Di dalamnya, pada baris ke-29 sampai baris ke-33 terdapat <TextInputEditText> dengan id @+id/etBillAmount yang digunakan untuk memasukkan nilai tagihan dan terhubung langsung dengan variabel etBillAmount pada Kotlin yang diinisialisasi pada baris ke-28.

Baris ke-28 sampai baris ke-31 pada Kotlin melakukan inisialisasi komponen menggunakan findViewById. Komponen spinnerTipPercentage terhubung dengan

<AutoCompleteTextView> pada XML baris ke-46 sampai baris ke-52 yang berada dalam <TextInputLayout> kedua pada baris ke-37 sampai baris ke-54. Komponen ini digunakan sebagai dropdown untuk memilih persentase tip.

Pada baris ke-33 sampai baris ke-35 dilakukan pengecekan if (savedInstanceState != null) untuk mengambil kembali nilai tipPercentageStr. Baris ke-37 mendefinisikan val tipOptions = listOf("15%", "18%", "20%"), kemudian pada baris ke-38 sampai baris ke-42 digunakan spinnerTipPercentage.post { ... }. Baris ke-39 membuat ArrayAdapter, baris ke-40 memasang adapter ke komponen dropdown XML, dan baris ke-41 mengatur nilai awal dengan setText(tipPercentageStr, false).

Pada baris ke-44 sampai baris ke-46 digunakan etBillAmount.doOnTextChanged { _, _, _, _ -> calculateTip() } sehingga setiap perubahan input pada <TextInputEditText> XML langsung memicu perhitungan. Baris ke-47 sampai baris ke-50 menggunakan setOnItemClickListener pada dropdown untuk memperbarui nilai tipPercentageStr, dan baris ke-51 sampai baris ke-53 menggunakan setOnCheckedChangeListener pada switch untuk mendeteksi perubahan status pembulatan.

Pada XML, bagian switch berada pada baris ke-55 sampai baris ke-73 dalam <LinearLayout>. Di dalamnya, pada baris ke-62 sampai baris ke-67 terdapat <TextView> dengan teks @string/round_up_tip, sedangkan pada baris ke-69 sampai baris ke-72 terdapat <SwitchMaterial> dengan id @+id/switchRoundUp yang terhubung dengan variabel switchRoundUp pada Kotlin.

Baris ke-54 pada Kotlin memanggil calculateTip() saat awal aplikasi dijalankan. Pada baris ke-57 sampai baris ke-60 terdapat fungsi onSaveInstanceState(outState: Bundle) yang menyimpan nilai tipPercentageStr menggunakan outState.putString("SAVED_TIP_PERCENT", tipPercentageStr).

Pada baris ke-61 dimulai fungsi calculateTip() sebagai inti logika. Baris ke-62 mengambil input dari etBillAmount, baris ke-63 mengambil status switch, baris ke-65 mengubah input menjadi angka menggunakan toDoubleOrNull() ?: 0.0, dan baris ke-66 mengambil nilai persentase dari string dengan tipPercentageStr.replace("%", ""). Baris ke-68 menghitung tip dengan (tipPercent / 100) * amount, kemudian pada baris ke-69 sampai

baris ke-71 dilakukan pembulatan menggunakan ceil(tip) jika switch aktif. Baris ke-73 memformat hasil dengan NumberFormat.getCurrencyInstance(Locale.US).format(tip), dan baris ke-74 menampilkan hasil ke tvTipAmount.

Pada bagian akhir XML, baris ke-75 sampai baris ke-83 terdapat <TextView> dengan id @+id/tvTipAmount yang digunakan untuk menampilkan hasil perhitungan tip dan terhubung dengan variabel tvTipAmount pada Kotlin. Seluruh komponen pada XML dihubungkan melalui id yang sama dengan yang digunakan pada Kotlin, sehingga setiap input, perubahan pilihan, dan status switch akan langsung diproses oleh fungsi calculateTip() dan hasilnya ditampilkan secara real-time pada tampilan.

Pembahasan Compose

MainActivity.kt:

Kode tersebut merupakan implementasi aplikasi Android untuk menghitung jumlah tip pembayaran menggunakan pustaka Jetpack Compose. Pada baris ke-1, deklarasi package com.example.tip berfungsi menentukan lokasi folder proyek di dalam struktur sistem. Selanjutnya, kumpulan perintah import dari baris ke-3 hingga ke-21 bertugas memuat berbagai pustaka bawaan yang dibutuhkan untuk menjalankan elemen antarmuka dan fungsi matematika pendukung. Pada baris ke-23, kode class MainActivity : ComponentActivity() { mendefinisikan kelas utama yang mengendalikan siklus hidup aplikasi. Di dalam kelas tersebut, baris ke-24 memuat override fun onCreate(savedInstanceState: Bundle?) { yang menjadi titik eksekusi pertama saat aplikasi dijalankan. Kemudian, baris ke-26 memanggil fungsi setContent { untuk mulai membangun tampilan visual, yang di dalamnya memuat komponen TipCalculatorScreen() pada baris ke-27.

Pusat antarmuka dan logika perhitungan berada pada fungsi TipCalculatorScreen di baris ke-34, yang diawali dengan penanda @Composable pada baris ke-33. Penyimpanan status data diatur mulai baris ke-35 menggunakan fungsi rememberSaveable yang disandingkan dengan mutableStateOf. Sebagai contoh, baris ke-35 memuat kode var amountInput by rememberSaveable { mutableStateOf("") } yang berfungsi menyimpan teks isian jumlah tagihan agar nilainya tidak hilang saat orientasi layar perangkat berubah. Baris ke-40 mendefinisikan daftar pilihan persentase tip melalui kode val tipOptions = listOf("15",

"18", "20"). Teks isian kemudian diubah menjadi format angka desimal pada baris ke-42 dengan kode val amount = amountInput.toDoubleOrNull() ?: 0.0. Perhitungan utama terjadi pada baris ke-45 melalui perintah var tip = (tipPercent / 100) * amount. Jika fitur pembulatan sedang aktif, baris ke-46 akan mengevaluasi kondisi if (roundUp), dan baris ke-47 menjalankan perintah tip = ceil(tip) untuk membulatkan nilai desimal ke atas. Nilai hasil perhitungan tersebut kemudian diformat menjadi gaya teks mata uang pada baris ke-50 dengan val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)

Penyusunan elemen visual dimulai pada baris ke-52 menggunakan komponen Column yang berfungsi menata seluruh elemen di dalamnya secara vertikal. Pada baris ke-59, komponen Text dibuat dengan atribut text = stringResource(R.string.calculate_tip) pada baris ke-60 yang berfungsi menampilkan judul utama pada bagian atas layar. Kotak isian untuk memasukkan jumlah tagihan dibangun mulai baris ke-66 menggunakan komponen TextField. Kotak ini langsung dihubungkan dengan variabel data pada baris ke-67 melalui kode value = amountInput serta baris ke-68 dengan kode onValueChange = { amountInput = it }. Agar tata letak papan ketik berubah menyesuaikan format masukan berupa angka, baris ke-76 menggunakan perintah keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number).

Pemilihan besaran persentase tip kemudian dikendalikan melalui menu lipat yang diawali pada baris ke-87 dengan komponen ExposedDropdownMenuBox. Komponen ini merespons status lipatan berdasarkan variabel di baris ke-88 dengan kode expanded = expanded. Di dalamnya, terdapat sebuah TextField pada baris ke-91 yang dibuat tidak bisa diketik secara manual karena pada baris ke-94 diatur dengan atribut readOnly = true. Saat komponen teks ini disentuh, daftar pilihan menu akan terbentang melalui pemanggilan komponen ExposedDropdownMenu pada baris ke-116. Komponen tersebut menyusun daftar opsi yang diambil dari variabel penampung melalui kode tipOptions.forEach { option -> pada baris ke-120. Setiap opsi diwakili oleh komponen DropdownMenuItem pada baris ke-121, yang apabila disentuh akan mengeksekusi blok kode pada baris ke-123 dengan perintah onClick = {, di mana perintah tersebut berfungsi memperbarui nilai persentase tip dan segera menutup kembali menu lipat.

Fitur pembulatan angka disajikan menggunakan tata letak horizontal yang dimulai pada baris ke-132 dengan komponen Row. Di dalam baris komponen tersebut, sebuah sakelar diimplementasikan pada baris ke-138 melalui komponen Switch. Sakelar interaktif ini terhubung langsung dengan variabel kendali melalui baris ke-139 dengan kode checked = roundUp dan memperbarui statusnya saat digeser melalui baris ke-140 dengan perintah onCheckedChange = { roundUp = it }. Setelah penyisipan jarak kosong menggunakan perintah Spacer(modifier = Modifier.height(16.dp)) pada baris ke-144, antarmuka ditutup dengan menampilkan hasil akhir kalkulasi. Angka akhir tersebut dimuat ke layar melalui komponen Text pada baris ke-146, di mana teks mata uang disisipkan menggunakan perintah text = stringResource(R.string.tip_amount, formattedTip) pada baris ke-147.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

Perbedaan Implementasi :

1. Arsitektur dan Struktur Kode : Implementasi pada XML memecah menjadi beberapa file, seperti untuk tampilan, untuk kode Kotlin untuk interaksi, dan asset lainnya. Sedangkan pada Jetpack Compose menggunakan deklaratif, seluruh UI akan dilakukan pada Kotlin tanpa file lain.
2. Adaptasi Layar : XML tampilan harus dibuat terpisah untuk ukuran layer tertentu, Sedangkan pada Jetpack Compose lebih fleksibel karena dapat menyesuaikan tampilan secara otomatis berdasarkan ukuran layer menggunakan fitur tertentu.
3. Penggunaan File dan Baris kode : membuat UI pada XML membutuhkan ode dengan baris yang banyak dan pada file yang berbeda beda. Sedangkan pada Jetpack Compose bisa lebih sedikit dan hanya satu file Kotlin.

Kelebihan XML :

1. Memiliki sistem yang sudah bagus bagi pengembang yang membutuhkan solusi kompabilitas ke perangkat android yang lama
2. Performa Scrolling yang baik dan mulus
3. Penggunaan baterai dan CPU yang lebih rendah pada beban tinggi
4. Pada penggunaan memori XML lebih konsisten

Kekurangan XML :

1. Pengembang harus secara manual menghubungkan status tampilan kode. Sehingga sering terjadi Bug.
2. Kurang Fleksibel untuk layar yang dinamis
3. Kode lebih panjang, seperti navigasi antar file yang akan memakan waktu dalam pengembangan

Kelibahan Jetpack Compose :

1. Pengelolaan status yang lebih efisien karena bisa otomatis dengan menambah fitur bawaan.
2. Performa rendering yang lebih baik untuk data yang sering diperbarui.
3. Produktivitas yang tinggi

Kekurangan Jetpack Compose :

1. Penggunaan memori yang lebih tinggi daripada XML
2. Mempunyai batas OS minimum Android 5.0 (API 21)
3. Beban CPU dan baterai pada aplikasi kompleks

SOAL 3

apa saja yang jadi daya jual Kotlin. Contoh (null safety, chain, elvish)

Daya jual Kotlin :

1. Fungsi Lambda, dalam Kotlin jika satu parameter, tidak perlu secara manual karena akan otomatis ada it. Tetapi juga ada peringatan karena bsia saja nested lambdas penggunaan parameter it secara berlebihan dan membuat kode susah untuk dibaca
2. Type Inference, ini akan sangat menambah efisiensi dari penulisan kode dan ringkas. Pada saat mendeklarasikan variable Kotlin akan otomatis menganggap bahwa variable ber tipe data apa.
3. Null Safety dan Elvish, di Kotlin biasanya terjadi error NullPointerException. Fitur safecall pada Kotlin efektif untuk melindungi aplikasi. Kotlin juga ada Elvish untuk menggantikan unsafe call agar terhindar dari error nullpointer.
4. Coroutines, memungkinkan pengguna Kotlin unutk mengeksekusi berbagai Tindakan secara bersamaan. Fitur ini sangat bagus untuk di jaman modern yang efisien

Daftar Pustaka

Mateus, B. G., & Martinez, M. (2020, October 5). On the adoption, usage and evolution of Kotlin features in Android development. *International Symposium on Empirical Software Engineering and Measurement*. https://doi.org/10.1145/3382494.3410676

Mayo, A., & Kilman, M. (n.d.). *A Study on XML vs. Compose for Dynamic UI Rendering in Robotics Control Apps: Creating Interfaces that Adapt to Different Phone Screen Sizes and Orientations*.

Pratama, R., Farisi, A., & Alamsyah, D. (n.d.). *4 TH MDP STUDENT CONFERENCE (MSC) 2025 Analisis Perbandingan Kinerja Aplikasi Android Java dan Kotlin dalam Mengonsumsi API*.

Singh, R., Rohit, E., Singh, K., & Singh, A. (2025). Comparative study of XML vs Jetpack Compose for UI Development in Android. | *IJIRT* |, *12*.

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

https://github.com/MuhammadNafisPutra/Praktikum-PemrogramanMobile.git